

RELACIÓN DE PROBLEMAS DE LA UD3 – CS2 DAW

Para depurar con VS code y PHP:

Se ha de comprobar si xdebug está instalado:

- Se revisa `phpinfo()` y se busca Xdebug
- Si aparece, está instalado, en otro caso hay que instalarlo

Descargar `php_xdebug-3.4.7-8.2-ts-vs16-x86_64.dll` y renombrarlo como `php_xdebug.dll` (el que os paso). Confirmar la dll adecuada.

Mover este fichero en `...\xampp\php\ext`

Cambiar `...\xampp\php\php.ini` y añadir esta línea: `zend_extension = xdebug`

Restaurar Apache

1. Instalación y uso de MySQL.

PHP ya incluye PDO_MySQL por defecto, pero hemos de verificarlo:

En `...\xampp\php\php.ini`. Tenemos que confirmar:

`;extension=pdo_mysql`

Si tiene ; al inicio, hay que eliminarlo para que quede: `extension=pdo_mysql`

Se ha de reiniciar Apache.

Hay que verificar que PDO está activo:

Creamos un archivo `phpinfo.php` en `...\xampp\htdocs` con:

```
<?php
phpinfo();
?>
```

En la información mostrada, buscaremos: PDO y `pdo_mysql`. Tiene que aparecer lo siguiente:

PDO support => enabled

PDO drivers => mysql, sqlLite

2. Instalación y uso de Oracle XE.

Se ha de configurar PHP para PDO con Oracle (OCI): necesitamos PDO_OCI o oci8.

- Descargar Oracle Instant Client: <https://www.oracle.com/database/technologies/instant-client/winx64-64-downloads.html>

Descargamos: Basic zip y SDK zip (necesario para PDO_OCI)

Se descomprime primero Basic.zip y de SDK se añaden los ficheros que hayan en la carpeta de instant pero no se actualiza Meta

- Instalar Instant Client:
Descomprimos ambos zips, por ejemplo: `C:\oracle\instantclient_23`
Se añade la ruta a la variable de entorno PATH: `C:\oracle\instantclient_23`
- Configuramos PHP:
En `...\xampp\php\php.ini`. Habilitamos las extensiones necesarias:
`extension=oci8_12c` ; para OCI8 con Oracle 12c/21c
`extension=pdo_oci` ; para PDO_OCI
Reiniciamos Apache.
- Verificamos el funcionamiento:
Creamos un archivo `phpinfo.php` en `...\xampp\htdocs` con:

```
<?php
phpinfo();
?>
```

En la información mostrada, buscaremos: PDO y pdo_oci. Tiene que aparecer lo siguiente:
OCI8 Support => enabled
PDO drivers => oci

3. Instalación y uso de ODBC.

ODBC (Open Database Connectivity) es una interfaz estándar que permite a las aplicaciones acceder a distintos sistemas gestores de bases de datos (MySQL, Oracle, SQL Server, etc.) mediante un driver ODBC específico para cada uno.

PHP puede usar ODBC de dos formas:

- Con la extensión ODBC nativa (odbc_connect, odbc_exec, etc.)
- Con PDO_ODBC, que usa el estándar PDO pero sobre un driver ODBC.

1. Instalar los drivers ODBC necesarios: para 64 bits.

- Para MySQL ODBC Driver. Descargamos MySQL ODBC driver (MySQL Connector/ODBC):
<https://dev.mysql.com/downloads/connector/odbc/>

Instálalo con el asistente (deja los valores por defecto).

- Para Oracle XE. Descargamos Oracle Instant Client y el paquete ODBC desde:
<https://www.oracle.com/database/technologies/instant-client/odbc-downloads.html>

Instalamos el Oracle Instant Client si no se ha realizado antes.

Descargamos el driver ODBC para Oracle.

Descomprimos el ZIP dentro del mismo directorio donde tenemos Instant Client instalado, por ejemplo: C:\oracle\instantclient_21_10

Añadimos esa carpeta al PATH:

En Opciones avanzadas → Variables de entorno
Edita la variable Path y agrega: C:\oracle\instantclient_21_10

Registra el driver ODBC ejecutando en PowerShell como administrador:

```
cd C:\oracle\instantclient_21_10
odbc_install.exe
```

Ha de mostrar un mensaje confirmando que el driver se registró correctamente.

2. Creamos las fuentes de datos ODBC (DSN): Los DSN permiten a ODBC saber a qué base de datos conectarse.

Abrimos el Administrador de orígenes de datos ODBC: 64 bits

Ve a la pestaña DSN de sistema (para todos los usuarios):

- Crear DSN para MySQL:

Agregar: MySQL ODBC x.x Unicode Driver.
Nombre del DSN: Por ejemplo MySQL_XAMPP
Servidor: localhost (el que sea)

Usuario: root (el que sea)
Base de datos: selecciona una (la que sea)
Pulsa Probar conexión: debería indicar éxito.

- Crear DSN para Oracle

Agregar: Oracle in instantclient_XX_X.
Nombre del DSN: Oracle_XE (el que se desee)
TNS Service Name: localhost/XEPDB1
Usuario: system (el que sea)
Pulsa Probar conexión: debería indicar éxito.

3. Activar la extensión PDO_ODBC en PHP (XAMPP):

Edita el fichero: ...\\xampp\\php\\php.ini

Descomenta o añade las líneas siguientes:

```
extension=php_pdo_odbc.dll  
extension=php_odbc.dll
```

Guarda y reinicia Apache desde el Panel de Control de XAMPP.

Comprueba con phpinfo.php que aparecen las secciones:

```
PDO_ODBC  
ODBC
```

Si aparecen, PHP ya puede usar DSN definidos en el sistema.

4. Ejemplos de conexión con ODBC en PHP

- PDO + ODBC con MySQL

```
<?php  
try {  
    $pdo = new PDO("odbc:MySQL_XAMPP", "root", "");  
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
    echo "Conexión ODBC con MySQL correcta";  
} catch (PDOException $e) {  
    echo "Error: " . $e->getMessage();  
}  
?>
```
- PDO + ODBC con Oracle XE

```
<?php  
try {  
    $pdo = new PDO("odbc:Oracle_XE", "system", "tu_contraseña");  
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
    echo "Conexión ODBC con Oracle XE correcta";  
} catch (PDOException $e) {  
    echo "Error: " . $e->getMessage();  
}  
?>
```
- Usando funciones ODBC directas (sin PDO)

```
<?php  
$conn = odbc_connect("MySQL_XAMPP", "root", "");  
if (!$conn) {
```

```

        echo "Error de conexión: " . odbc_errormsg();
    } else {
        echo "Conectado mediante ODBC a MySQL";
        odbc_close($conn);
    }
    ?>

```

4. A partir de los siguientes parámetros de conexión, realizar el código necesario para conectarse a la BD Oracle y obtener los empleados almacenados.

- Parámetros para PDO:
 - dsn: 'oci:dbname=IP a considerar/XE;charset=UTF8'
 - usuario: 'c##usu_ex_al1'
 - password: 'EX2526_AL1'
- Consideraciones:
 - Se ha de utilizar un fichero de configuración en el que se almacenarán dichos parámetros.
 - En el constructor se han de leer e instanciar en la clase estos parámetros del fichero de configuración.
 - Se ha de utilizar para las sentencias SQL con parámetros seguros (la opción es opcional).
 - Lanzar la siguiente consulta SQL: SELECT * FROM empleados y mostrar el nombre y apellido 1 de cada fila.
 - Para la creación de la tabla se ha utilizado esta sentencia:

```

CREATE TABLE empleados (
    num_empleado NUMBER(6) NOT NULL,
    dni          VARCHAR2(9) NOT NULL,
    nombre       VARCHAR2(50) NOT NULL,
    apellido1    VARCHAR2(50) NOT NULL,
    apellido2    VARCHAR2(50) NOT NULL,
    tfno_empresa NUMBER(9) NOT NULL,
    sueldo       NUMBER(9,2) NOT NULL,
    tfno_personal NUMBER(9) NULL,
    transporte   NUMBER (6,2) NULL,
    num_jefe     NUMBER(6) NULL,
    cantidad_contactos NUMBER(3) NULL
);

```

```

ALTER TABLE empleados ADD CONSTRAINT emp_pk PRIMARY KEY ( num_empleado );
ALTER TABLE empleados ADD CONSTRAINT emp_un1 UNIQUE ( dni );

```

```

ALTER TABLE empleados
    ADD CONSTRAINT emp_fk1 FOREIGN KEY ( num_jefe )
        REFERENCES empleados ( num_empleado );

```

```

CREATE INDEX emp_idx1 ON empleados (nombre ASC, apellido1 ASC, apellido2 ASC);

```

- Controlar las excepciones necesarias.

5. A partir de los siguientes parámetros de conexión, realizar el código necesario para conectarse a la BD MySql y obtener los empleados almacenados.

- Parámetros para PDO:
 - dsn: 'mysql:host=IP al que conectarse;dbname=referencia;charset=utf8mb4'
 - usuario: 'c1bd_admin'
 - password: 'BDA1'
- Consideraciones:
 - Se ha de utilizar un fichero de configuración en el que se almacenarán dichos parámetros.

- En el constructor se han de leer e instanciar en la clase estos parámetros del fichero de configuración.
- Se ha de utilizar para las sentencias SQL con parámetros seguros (la opción es opcional).
- Lanzar la siguiente consulta SQL: SELECT * FROM empleados y mostrar el nombre y apellido 1 de cada fila.
- Para la creación de la tabla se ha utilizado esta sentencia:

```
CREATE TABLE empleados (
    num_empleado INT NOT NULL,
    dni          VARCHAR(9) NOT NULL,
    nombre       VARCHAR(50) NOT NULL,
    apellido1    VARCHAR(50) NOT NULL,
    apellido2    VARCHAR(50) NOT NULL,
    tfno_empresa INT NOT NULL,
    sueldo       DECIMAL(9,2) NOT NULL,
    tfno_personal INT NULL,
    transporte   DECIMAL(6,2) NULL,
    num_jefe     INT NULL
);
```

```
ALTER TABLE empleados ADD CONSTRAINT emp_pk PRIMARY KEY (num_empleado);
ALTER TABLE empleados ADD CONSTRAINT emp_un1 UNIQUE (dni);
ALTER TABLE empleados
    ADD CONSTRAINT emp_fk1 FOREIGN KEY (num_jefe)
        REFERENCES empleados (num_empleado);
CREATE INDEX emp_idx1 ON empleados (nombre, apellido1, apellido2);
```

- Controlar las excepciones necesarias.

6. A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD Oracle e insertar una fila en la tabla de empleados anterior. Realizarlo sin transacciones.

- Consideraciones:
 - Se ha de utilizar un fichero de configuración en el que se almacenarán dichos parámetros.
 - En el constructor se han de leer e instanciar en la clase estos parámetros del fichero de configuración.
 - Se ha de utilizar para las sentencias SQL con parámetros seguros (la opción es opcional).
 - Insertar la siguiente información:
 - num_empleado: 100
 - dni: 12345678X
 - nombre: Nombre6
 - apellido1: Apellido16
 - apellido2: Apellido26
 - tfno_empresa: 999100
 - sueldo: 30000
 - tfno_personal: 555000
 - transporte: 2500
 - num_jefe: 1

```
<?php
declare(strict_types=1);

class OracleConexion
{
    // para config.php
    private string $dsn;
    private string $usuario;
    private string $password;
```

```

private ?PDO $conexion = null;

//Constructor de la conexión
// si utilizamos un config.php
public function __construct(string $rutaConfig = './secure/xe_config.php' ) {
    if (!file_exists($rutaConfig)) {
        throw new Exception("No se encontró el archivo de configuración: $rutaConfig");
    }

    // incluimos el archivo y espera que devuelva un array asociativo
    // en realidad, estamos utilizando un array con los parámetros de conexión.
    $config = require $rutaConfig;

    // si no es un array o no están en el array los parámetros que necesitamos
    if (!is_array($config) ||
        !isset($config['dsn'], $config['usuario'], $config['password'])) {
        throw new Exception("El archivo de configuración no contiene los parámetros esperados.");
    }

    $this->dsn = $config['dsn'];
    $this->usuario = $config['usuario'];
    $this->password = $config['password'];
}

//Establece la conexión con la base de datos Oracle
public function conectar(): bool
{
    try {
        $this->conexion = new PDO($this->dsn, $this->usuario, $this->password);
        $this->conexion->setAttribute(PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION);

        return true;
    } catch (PDOException $e) {
        echo "Error de conexión: " . $e->getMessage();
        return false;
    }
}

//Devuelve la conexión PDO
public function getConexion(): ?PDO
{
    return $this->conexion;
}

// Cierra la conexión
public function desconectar(): void
{
    $this->conexion = null; //Cierra la conexión explícitamente
    echo "Conexión cerrada correctamente". "<br>";
}

//Ejecuta una sentencia INSERT, UPDATE o DELETE
public function ejecutarSentencia(string $sql, array $parametros = []): int
{
    if ($this->conexion === null) {
        throw new PDOException("No hay conexión activa con la base de datos.");
    }
}

```

```

        $stmt = $this->conexion->prepare($sql);
        $stmt->execute($parametros);
        return $stmt->rowCount();
    }
}

// probamos el código
// creamos la conexión
try {
    $oracle = new OracleConexion( "./secure/xe_config.php");

    if ($oracle->conectar()) {
        echo "Conexión establecida correctamente.<br>";

        // realizamos el procesamiento
        $stmt = "INSERT INTO EMPLEADOS
            (num_empleado, dni, nombre, apellido1, apellido2,
            tfno_empresa, sueldo, tfno_personal, transporte, num_jefe)
            VALÚES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

        $parametros=[100,
            '12345678X',
            'Nombre6',
            'Apellido16',
            'Apellido26',
            999100,
            30000,
            555000,
            2500,
            1 ];
        $oracle->ejecutarSentencia($stmt, $parametros);

        echo "Los datos han sido insertados correctamente". "<br>";

        $oracle->desconectar();
    }
} catch (PDOException $e) {
    echo "Error de base de datos: " . $e->getMessage();
} catch (Exception $e) {
    echo "Error general: " . $e->getMessage();
}
?>

```

7. (Entregable) A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD MySql e insertar una fila en la tabla de empleados anterior. Realizarlo sin transacciones y con una transacción. Las consideraciones e información a insertar es la misma que la del ejercicio 6.

8. A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD Oracle para modificar una fila en la tabla de empleados anterior. Realizarlo sin transacciones y con una transacción.

- Consideraciones:
 - Se ha de utilizar un fichero de configuración en el que se almacenarán dichos parámetros.
 - En el constructor se han de leer e instanciar en la clase estos parámetros del fichero de configuración.

- Se ha de utilizar para las sentencias SQL con parámetros seguros (la opción es opcional).
- Modificar la siguiente información:
 - num_empleado: 100
 - nombre: Jose
 - apellido1: Garcia
 - apellido2: Garcia
- Con transacción indicar los datos en mayúscula.

```
<?php
declare(strict_types=1);

class OracleConexion
{
    // para config.php
    private string $dsn;
    private string $usuario;
    private string $password;
    private ?PDO $conexion = null;

    //Constructor de la conexión
    // si utilizamos un config.php
    public function __construct(string $rutaConfig = './secure/xe_config.php' ) {
        if (!file_exists($rutaConfig)) {
            throw new Exception("No se encontró el archivo de configuración: $rutaConfig");
        }

        // incluimos el archivo y espera que devuelva un array asociativo
        // en realidad, estamos utilizando un array con los parámetros de conexión.
        $config = require $rutaConfig;

        // si no es un array o no están en el array los parámetros que necesesitamos
        if (!is_array($config) ||
            !isset($config['dsn'], $config['usuario'], $config['password'])) {
            throw new Exception("El archivo de configuración no contiene los parámetros esperados.");
        }

        $this->dsn = $config['dsn'];
        $this->usuario = $config['usuario'];
        $this->password = $config['password'];
    }

    //Establece la conexión con la base de datos Oracle
    public function conectar(): bool
    {
        try {
            $this->conexion = new PDO($this->dsn, $this->usuario, $this->password);
            $this->conexion->setAttribute(PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION);

            return true;
        } catch (PDOException $e) {
            echo "Error de conexión: " . $e->getMessage();
            return false;
        }
    }

    //Devuelve la conexión PDO
    public function getConexion(): ?PDO
```



```

{
    return $this->conexion;
}

// Cierra la conexión
public function desconectar(): void
{
    $this->conexion = null; //Cierra la conexión explícitamente
    echo "Conexión cerrada correctamente". "<br>";
}

//Ejecuta una sentencia INSERT, UPDATE o DELETE
public function ejecutarSentencia(string $sql, array $parametros = []): int
{
    if ($this->conexion === null) {
        throw new PDOException("No hay conexión activa con la base de datos.");
    }

    $stmt = $this->conexion->prepare($sql);
    $stmt->execute($parametros);
    return $stmt->rowCount();
}

// Ejecuta una sentencia INSERT, UPDATE o DELETE con transacción
public function ejecutarSentenciaT(string $sql, array $parametros = []): int
{
    if ($this->conexion === null) {
        throw new PDOException("No hay conexión activa con la base de datos.");
    }

    $stmt = null;

    try {
        // Iniciar transacción
        $this->conexion->beginTransaction();

        // Preparar y ejecutar
        $stmt = $this->conexion->prepare($sql);
        $stmt->execute($parametros);

        // Obtener filas afectadas
        $filas = $stmt->rowCount();

        // Confirmar si tod va bien
        $this->conexion->commit();

        // Liberar statement
        $stmt = null;

        return $filas;
    } catch (PDOException $e) {
        // Deshacer si todavía está la transacción activa
        if ($this->conexion->inTransaction()) {
            $this->conexion->rollBack();
        }

        // Liberar statement y relanzar la excepción
        $stmt = null;
    }
}

```

```

        throw $e; // relanzamos la excepción para que el código que nos haya llamado
                // conozca el error. Si no, se queda en la función y no se transmite
    }
}
}

// probamos el código
// creamos la conexión
try {
    $oracle = new OracleConexion( "./secure/xe_config.php");

    if ($oracle->conectar()) {
        echo "Conexión establecida correctamente.<br>";

        // realizamos el procesamiento sin transacción
        $stmt = "UPDATE EMPLEADOS
                SET nombre= ?,
                    apellido1 = ?,
                    apellido2=?
                WHERE num_empleado = ?";

        $parametros=['Jose',
                    'Garcia',
                    'Garcia',
                    100 ];

        $oracle->ejecutarSentencia($stmt, $parametros);

        echo "Los datos han sido insertados correctamente". "<br>";

        // realizamos el procesamiento con transacción
        $stmt = "UPDATE EMPLEADOS
                SET nombre= ?,
                    apellido1 = ?,
                    apellido2=?
                WHERE num_empleado = ?";

        $parametros=['JOSE',
                    'GARCIA',
                    'GARCIA',
                    100 ];

        $oracle->ejecutarSentenciaT($stmt, $parametros);

        echo "Los datos han sido insertados correctamente con transacción". "<br>";
        $oracle->desconectar();
    }
} catch (PDOException $e) {
    echo "Error de base de datos: " . $e->getMessage();
} catch (Exception $e) {
    echo "Error general: " . $e->getMessage();
}
?>

```

9. (Entregable) A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD MySql para modificar una fila en la tabla de empleados anterior. Realizarlo sin transacciones y con una transacción. Las consideraciones e información a insertar es la misma que la del ejercicio 8.

10. A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD Oracle para eliminar una fila en la tabla de empleados anterior. Realizarlo sin transacciones y con una transacción.

- Consideraciones:
 - Se ha de utilizar un fichero de configuración en el que se almacenarán dichos parámetros.
 - En el constructor se han de leer e instanciar en la clase estos parámetros del fichero de configuración.
 - Se ha de utilizar para las sentencias SQL con parámetros seguros (la opción es opcional).
 - Eliminar la siguiente información:
 - num_empleado: 100

El código a lanzar sería este:

```
...
// probamos el código
// creamos la conexión
try {
    $oracle = new OracleConexion( "./secure/xe_config.php");

    if ($oracle->conectar()) {
        echo "Conexión establecida correctamente.<br>";

        // realizamos el procesamiento sin transacción
        $stmt = "DELETE EMPLEADOS
                WHERE num_empleado = ?";

        $parametros=[100];

        $filas = $oracle->ejecutarSentencia($stmt, $parametros);

        echo "Los datos han sido eliminados correctamente, # afectadas: ($filas)".
            "<br>";

        // realizamos el procesamiento con transacción
        $filas = $oracle->ejecutarSentenciaT($stmt, $parametros);

        echo "Los datos han sido eliminados correctamente con transacción, # afectadas: ($filas)".
            "<br>";

        $oracle->desconectar();
    }
} catch (PDOException $e) {
    echo "Error de base de datos: " . $e->getMessage();
} catch (Exception $e) {
    echo "Error general: " . $e->getMessage();
}
```

11. (Entregable) A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD MySql para eliminar una fila en la tabla de empleados anterior.

Realizarlo sin transacciones y con una transacción. Las consideraciones e información a insertar es la misma que la del ejercicio 10.

12. A partir de los parámetros de conexión de los ejercicios previos, realizar el código necesario para conectarse a la BD Oracle para ejecutar el siguiente procedimiento almacenado.

13. A partir de los siguientes parámetros de conexión, realizar el código necesario para conectarse a la BD MySql para ejecutar el siguiente procedimiento almacenado.

14. Modelar la implementación de la interfaz siguiente: para conectarnos a la BD Oracle utilizada previamente.

- ConexionBD (interfaz):
 - Con estos métodos:
 - conectarBD.
 - desconectarBD.
- ConexionPDO_Oracle (implementación de la interfaz con PDO):
 - Con estos atributos:
 - dsn, cadena.
 - usuario, cadena.
 - pwd, cadena.
 - conexión, PDO.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación.
 - Obtener la conexión con los métodos get.
 - setParmetros que reinicie los valores de conexión.
 - conectarBD: de un fichero de conexión, tal y como lo hemos realizado en ejercicios anteriores.
 - conectarBD_Valores: realiza la conexión pasándole la información de los parámetros.
 - desconectarBD.
- ConexionPDO_Mysql (implementación de la interfaz con PDO):
 - Con estos atributos:
 - dsn, cadena.
 - usuario, cadena.
 - pwd, cadena.
 - conexión, PDO.
 - Con estos métodos:
 - Constructor de la clase que reciba los valores iniciales del mismo en su creación.
 - Obtener la conexión con los métodos get.
 - setParmetros que reinicie los valores de conexión.
 - conectarBD: de un fichero de conexión, tal y como lo hemos realizado en ejercicios anteriores.
 - conectarBD_Valores: realiza la conexión pasándole la información de los parámetros.
 - desconectarBD.

Implementar en PHP las clases con lo necesario para poder probar su ejecución y prueba.